UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Department of Computer Science – Software Engineering

ADVANCED WEB DEVELOPMENT

Names: Esteban Quiroga, Angel Sabando, David Rodriguez

NRC: 30716

Date: May 24, 2026

Full System Documentation: ATS Express

System Endpoints & Resource Architecture

PART 1

Backend API Endpoints

Server-side routes · JSON responses · https://backend-api.onrender.com/api

Overview

This document contains the complete URI mapping for the entire ATS Express system. It covers two main areas:

- **Part 1 — Backend API Endpoints**: Server-side routes that handle data processing and return JSON responses.
- **Part 2 — Frontend Page URIs**: Client-side application routes that render the user interface.

Base URL

The base URL for all Backend API requests is:

```
https://backend-api.onrender.com/api
```

Endpoints

POST /users/login

Authenticates a user and returns their information.

POST `/users/login`

Parameters

- **email** (required): The email address of the user.
- **password** (required): The password of the user.

Response

Returns a JSON object with the following properties:

- **success**: Boolean indicating if login was successful.
- **data**: An object with user details: id, ruc, firstName, lastName, email, role.
- **message**: A brief message about the operation result.

Example — Request

```
POST /users/login
```

```
{
    "email": "user@example.com",
    "password": "password123"
}
```

Example — Response

```
{
    "success": true,
    "data": {
        "id": "07787dd8-aafa-4c6d-a49c-07595438199d",
        "ruc": "1790011223002",
        "firstName": "John",
        "lastName": "Doe",
        "email": "user@example.com",
        "role": "user",
        "createdAt": "2026-05-24T10:00:00Z"
    },
    "message": "Login successful"
}
```

POST /users/register

Registers a new user in the platform.

POST `/users/register`

Parameters

- **ruc** (required): The RUC of the user (must be exactly 13 numeric digits).
- **firstName** (required): The first name of the user (cannot contain numbers).
- **lastName** (required): The last name of the user (cannot contain numbers).
- **email** (required): The email of the user.
- **password** (required): The password for the new account.

Response

- **success**: Boolean indicating if registration was successful.
- **data**: The newly created user object.
- **message**: A brief message about the operation result.

Example — Request

```
POST /users/register

{
    "ruc": "1790011223002",
    "firstName": "Jane",
    "lastName": "Doe",
    "email": "jane@example.com",
    "password": "securepassword123"
}
```

Example — Response

```
{
    "success": true,
    "data": {
        "id": "18898ee9-bbgb-5d7e-b50d-18606549200e",
        "ruc": "1790011223002",
        "firstName": "Jane",
        "lastName": "Doe",
        "email": "jane@example.com",
        "role": "user",
        "createdAt": "2026-05-24T10:15:00Z"
    },
    "message": "User registered successfully"
}
```

GET /users

Returns a list of all registered users in the platform.

GET `/users`

Parameters

None.

Response

- **success**: Boolean indicating if the request was successful.
- **data**: An array of user objects.

Example — Request

```
GET /users
```

Example — Response

```
{
    "success": true,
    "data": [
        {
            "id": "07787dd8-aafa-4c6d-a49c-07595438199d",
            "ruc": "1790011223002",
            "firstName": "John",
            "lastName": "Doe",
            "email": "user@example.com",
            "role": "user",
            "createdAt": "2026-05-24T10:00:00Z"
        },
        ...
    ]
}
```

POST /users/reset-password

Resets a user's password based on their email.

POST `/users/reset-password`

Parameters

- **email** (required): The email address of the user.
- **newPassword** (required): The new password to be set.

Response

- **success**: Boolean indicating if the update was successful.
- **data**: Null.
- **message**: A brief message about the operation result.

Example — Request

```
POST /users/reset-password

{
    "email": "user@example.com",
    "newPassword": "newsecurepassword123"
}
```

Example — Response

```
{
    "success": true,
    "data": null,
    "message": "Password updated successfully"
}
```

POST /users/update

Updates user information (First Name, Last Name, and Email).

POST `/users/update`

Parameters

- **id** (required): The ID of the user to update.
- **firstName** (optional): The new first name.
- **lastName** (optional): The new last name.
- **email** (optional): The new email.

Response

Returns a JSON object indicating if the update was successful.

Example — Request

```
POST /users/update

{
    "id": "07787dd8-aafa-4c6d-a49c-07595438199d",
    "firstName": "John updated",
```

```
    "lastName": "Doe updated"
}
```

Example — Response

```
{
    "success": true,
    "data": null,
    "message": "User updated successfully"
}
```

POST /users/delete

Deletes a user from the system.

POST `/users/delete`

Parameters

- **id** (required): The ID of the user to delete.

Response

Returns a JSON object indicating if the deletion was successful.

Example — Request

```
POST /users/delete

{
    "id": "07787dd8-aafa-4c6d-a49c-07595438199d"
}
```

Example — Response

```
{
    "success": true,
    "data": null,
    "message": "User deleted successfully"
}
```

GET /dashboard/{id}

Returns a summary of statistics and notifications for a specific user's dashboard.

GET `/dashboard/{id}`

Parameters

- **id** (required): The user's ID passed in the URL path.

Response

- **success**: Boolean indicating if the request was successful.
- **data**: An object with dashboard metrics: invoicesDownloaded, invoicesDownloadedChange, errorsDetected, lastSync, notifications.

Example — Request

```
GET /dashboard/07787dd8-aafa-4c6d-a49c-07595438199d
```

Example — Response

```
{
    "success": true,
    "data": {
        "invoicesDownloaded": 120,
        "invoicesDownloadedChange": 15,
        "errorsDetected": 3,
        "lastSync": "2026-05-24 14:00:00",
        "notifications": [
            {
                "id": 1,
                "type": "warning",
                "title": "Notice",
                "message": "Review invoices with errors",
                "timestamp": "2026-05-24 14:00:00",
                "read": false
            }
        ]
    }
}
```

POST /support/tickets

Creates a new support ticket.

POST `/support/tickets`

Parameters

- **userId** (optional): The ID of the user creating the ticket.
- **subject** (required): The subject or brief description of the issue.

- **category** (required): The category of the support request (e.g., 'sri-connection', 'account').
- **priority** (optional): Priority of the ticket ('low', 'medium', 'high'). Defaults to 'medium'.
- **description** (required): Detailed description of the problem (must be at least 20 characters).

Response

- **success**: Boolean indicating if the ticket was created.
- **data**: An object containing the generated ticket number.
- **message**: A brief message about the operation result.

Example — Request

```
POST /support/tickets

{
    "userId": "07787dd8-aafa-4c6d-a49c-07595438199d",
    "subject": "Cannot download XML invoices",
    "category": "invoice-download",
    "priority": "high",
    "description": "I have connected my SRI account successfully but when I
click download, nothing happens for several minutes."
}
```

Example — Response

```
{
    "success": true,
    "data": {
        "ticketNumber": "#TKT-2026-a1b2"
    },
    "message": "Ticket submitted successfully"
}
```

Error Codes (Backend API)

This API uses the following error codes:

HTTP Code	Description
`400 Bad Request`	The request was malformed, missing required parameters, or failed validation.
`401 Unauthorized`	The credentials provided were invalid.
`404 Not Found`	The requested resource or user was not found.
`500 Internal Server Error`	An unexpected error occurred on the server.

PART 2

Frontend Page URIs

Client-side routes · Browser pages · https://ats-express.com

Overview

These are the visual pages navigated by the user in the browser. They do not return JSON, but rather render the application interfaces.

Base URL

The application is accessed via the frontend domain:

```
https://ats-express.com
http://localhost:5173   (development)
```

Public Pages

No Session Required

Route	Component	Description
`GET /`	`LandingPage`	The main promotional landing page.
`GET /login`	`LoginPage`	Authentication screen for existing users.
`GET /register`	`RegisterPage`	Registration screen for new users.
`GET /forgot-password`	`ForgotPasswordPage`	Password recovery interface.

Protected Pages

Session Required

Route	Component	Description
`GET /dashboard`	`DashboardPage`	User control panel with metrics and quick actions.
`GET /sri-connection`	`SriConnectionPage`	Form to link the user's account with the SRI web services.

`GET /invoices/download`	`InvoicesDownloadPage`	Interface to trigger automatic invoice downloads.
`GET /invoices/manage`	`InvoicesManagePage`	Data table listing all invoices with viewing/editing options.
`GET /ats/generate`	`AtsGeneratePage`	Generates the ATS logic in XLSM macro format.
`GET /ats/validate`	`AtsValidatePage`	Analyzes local data against SRI constraints and shows errors.
`GET /ats/export`	`AtsExportPage`	Exports the final approved ATS into XML format.
`GET /traceability`	`TraceabilityPage`	Audit log and history of user actions (downloads, exports).
`GET /support`	`SupportPage`	Support ticket submission form.
`GET /profile`	`ProfilePage`	User settings and profile management.

Admin Protected Pages

Admin Role Required

Route	Component	Description
`GET /admin/dashboard`	`AdminDashboardPage`	System overview and global statistics for administrators.
`GET /admin/users`	`AdminUsersPage`	User management table (Create, Read, Update, Delete users).
`GET /admin/audit`	`AdminAuditPage`	Global audit trail showing actions from all system users.
`GET /admin/settings`	`AdminSettingsPage`	General system configurations and parameters.